Design and Analysis of Algorithms

SLIP 1

Q.1) Write a program to sort a list of n numbers in ascending order using selection sort and
determine the time required to sort the elements .

```
#include <stdio.h>
#include <time.h>

void selectionSort(int arr[], int n) {
    int i, j, min_idx, temp;

    for (i = 0; i < n - 1; i++) {
        min_idx = i;

        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }

        temp = arr[i];
        arr[i] = arr[min_idx];
        arr[min_idx] = temp;
    }
}

int main() {
    int n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    clock_t start = clock();
    selectionSort(arr, n);
    clock_t end = clock();

    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("\nTime taken: %f seconds\n", time_taken);

    return 0;
}
```

Q.2) Write a program to sort a given set of elements using the Quick sort method and determine
the time required to sort the elements. Repeat the experiment for different values of n, the
number of elements in the list to be sorted. The elements can be read from a file or can be
generated using the random number generator.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivot = arr[high];
        int i = low - 1, j, temp;

        for (j = low; j < high; j++) {
            if (arr[j] <= pivot) {
                i++;
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;

        int pi = i + 1;

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    srand(time(0));
    for (i = 0; i < n; i++) {
        arr[i] = rand() % 1000;
    }

    clock_t start = clock();
    quickSort(arr, 0, n - 1);
    clock_t end = clock();

    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("\nTime taken: %f seconds\n", time_taken);

    return 0;
}
```

SLIP 2

Q.1) Write a program to sort n randomly generated elements using Heapsort method.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;
    int temp;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    int i, temp;

    for (i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (i = n-1; i > 0; i--) {
        temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
    }
}

int main() {
    int n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
    srand(time(0));

    for (i = 0; i < n; i++) {
```

```
        arr[i] = rand() % 1000;
    }

    clock_t start = clock();
    heapSort(arr, n);
    clock_t end = clock();

    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("\nTime taken: %f seconds\n", time_taken);

    return 0;
}
```

Q.2) Write a program to implement Strassenâ ™s Matrix multiplication

```
#include <stdio.h>

void add(int A[2][2], int B[2][2], int C[2][2]) {
    for(int i=0;i<2;i++)
        for(int j=0;j<2;j++)
            C[i][j] = A[i][j] + B[i][j];
}

void sub(int A[2][2], int B[2][2], int C[2][2]) {
    for(int i=0;i<2;i++)
        for(int j=0;j<2;j++)
            C[i][j] = A[i][j] - B[i][j];
}

void multiply(int A[2][2], int B[2][2], int C[2][2]) {
    int M1, M2, M3, M4, M5, M6, M7;

    M1 = (A[0][0]+A[1][1])*(B[0][0]+B[1][1]);
    M2 = (A[1][0]+A[1][1])*B[0][0];
    M3 = A[0][0]*(B[0][1]-B[1][1]);
    M4 = A[1][1]*(B[1][0]-B[0][0]);
    M5 = (A[0][0]+A[0][1])*B[1][1];
    M6 = (A[1][0]-A[0][0])*(B[0][0]+B[0][1]);
    M7 = (A[0][1]-A[1][1])*(B[1][0]+B[1][1]);

    C[0][0] = M1 + M4 - M5 + M7;
    C[0][1] = M3 + M5;
    C[1][0] = M2 + M4;
    C[1][1] = M1 - M2 + M3 + M6;
}

int main() {
    int A[2][2], B[2][2], C[2][2];

    printf("Enter Matrix A (2x2):\n");
    for(int i=0;i<2;i++)
        for(int j=0;j<2;j++)
            scanf("%d",&A[i][j]);

    printf("Enter Matrix B (2x2):\n");
    for(int i=0;i<2;i++)
```

```
        for(int j=0;j<2;j++)
            scanf("%d",&B[i][j]);

    multiply(A,B,C);

    printf("Result Matrix:\n");
    for(int i=0;i<2;i++) {
        for(int j=0;j<2;j++)
            printf("%d ",C[i][j]);
        printf("\n");
    }

    return 0;
}
```

SLIP 3

Q.1) Write a program to sort a given set of elements using the Quick sort method and determine
the time required to sort the elements.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivot = arr[high];
        int i = low - 1, j, temp;

        for (j = low; j < high; j++) {
            if (arr[j] <= pivot) {
                i++;
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;

        int pi = i + 1;

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
```

```
    srand(time(0));
    for (i = 0; i < n; i++) {
        arr[i] = rand() % 1000;
    }

    clock_t start = clock();
    quickSort(arr, 0, n - 1);
    clock_t end = clock();

    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("\nTime taken: %f seconds\n", time_taken);

    return 0;
}
```

Q.2) Write a program to find Minimum Cost Spanning Tree of a given undirected graph using
Prims algorithm

```
#include <stdio.h>
#include <limits.h>

#define V 10

int minKey(int key[], int mstSet[], int n) {
    int min = INT_MAX, min_index;

    for (int v = 0; v < n; v++)
        if (mstSet[v] == 0 && key[v] < min)
            min = key[v], min_index = v;

    return min_index;
}

void primMST(int graph[V][V], int n) {
    int parent[V];
    int key[V];
    int mstSet[V];

    for (int i = 0; i < n; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }

    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < n - 1; count++) {
        int u = minKey(key, mstSet, n);
        mstSet[u] = 1;

        for (int v = 0; v < n; v++)
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
```

```
            }
    }

    int total = 0;
    printf("Edge \tWeight\n");
    for (int i = 1; i < n; i++) {
        printf("%d - %d \t%d\n", parent[i], i, graph[i][parent[i]]);
        total += graph[i][parent[i]];
    }
    printf("Total Cost = %d\n", total);
}

int main() {
    int n;
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    int graph[V][V];

    printf("Enter adjacency matrix:\n");
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            scanf("%d",&graph[i][j]);

    primMST(graph, n);

    return 0;
}
```

SLIP 4

Q.1) Write a program to implement a Merge Sort algorithm to sort a given set of elements and
determine the time required to sort the elements

```
#include <stdio.h>
#include <time.h>

void merge(int arr[], int l, int m, int r) {
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[n1], R[n2];

    for (i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    i = j = 0;
    k = l;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
```

```
        else
            arr[k++] = R[j++];
    }

    while (i < n1)
        arr[k++] = L[i++];

    while (j < n2)
        arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    clock_t start = clock();
    mergeSort(arr, 0, n - 1);
    clock_t end = clock();

    printf("Sorted array:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("\nTime taken: %f seconds\n", time_taken);

    return 0;
}
```

Q.2) Write a program to implement Knapsack problems using Greedy method

```
#include <stdio.h>

struct Item {
    int value, weight;
    float ratio;
};

void sort(struct Item arr[], int n) {
    for(int i=0;i<n-1;i++) {
        for(int j=i+1;j<n;j++) {
            if(arr[i].ratio < arr[j].ratio) {
                struct Item temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
```

```
}

int main() {
    int n, capacity;

    printf("Enter number of items: ");
    scanf("%d", &n);

    struct Item arr[n];

    printf("Enter values:\n");
    for(int i=0;i<n;i++)
        scanf("%d", &arr[i].value);

    printf("Enter weights:\n");
    for(int i=0;i<n;i++) {
        scanf("%d", &arr[i].weight);
        arr[i].ratio = (float)arr[i].value / arr[i].weight;
    }

    printf("Enter capacity: ");
    scanf("%d", &capacity);

    sort(arr, n);

    float total = 0;

    for(int i=0;i<n;i++) {
        if(capacity >= arr[i].weight) {
            capacity -= arr[i].weight;
            total += arr[i].value;
        } else {
            total += arr[i].ratio * capacity;
            break;
        }
    }

    printf("Maximum value = %.2f\n", total);

    return 0;
}
```

SLIP 5

Q.1) Write a program for the Implementation of Kruskalâ ™s algorithm to find minimum
cost spanning tree.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

struct Edge {
    int u, v, weight;
};

struct Edge edges[MAX];
```

```
int parent[MAX];

int find(int i) {
    while (parent[i] != i)
        i = parent[i];
    return i;
}

void union_set(int u, int v) {
    int u_root = find(u);
    int v_root = find(v);
    parent[u_root] = v_root;
}

void sortEdges(int e) {
    for (int i = 0; i < e - 1; i++) {
        for (int j = 0; j < e - i - 1; j++) {
            if (edges[j].weight > edges[j + 1].weight) {
                struct Edge temp = edges[j];
                edges[j] = edges[j + 1];
                edges[j + 1] = temp;
            }
        }
    }
}

int main() {
    int V, E;

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter number of edges: ");
    scanf("%d", &E);

    for (int i = 0; i < E; i++) {
        printf("Enter edge %d (u v weight): ", i + 1);
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].weight);
    }

    for (int i = 0; i < V; i++) {
        parent[i] = i;
    }

    sortEdges(E);

    int count = 0, i = 0;
    int total_cost = 0;

    printf("\nEdges in MST:\n");

    while (count < V - 1 && i < E) {
        int u = edges[i].u;
        int v = edges[i].v;

        int set_u = find(u);
        int set_v = find(v);

        if (set_u != set_v) {
            printf("%d -- %d == %d\n", u, v, edges[i].weight);
            total_cost += edges[i].weight;
            union_set(set_u, set_v);
            count++;
        }
```

```
        i++;
    }

    printf("Total cost of MST = %d\n", total_cost);

    return 0;
}
```

Q.2) Write a program to implement Huffman Code using greedy methods and also calculate the
best case and worst-case complexity.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

struct Node {
    char data;
    unsigned freq;
    struct Node *left, *right;
};

struct MinHeap {
    int size;
    struct Node* array[MAX];
};

struct Node* newNode(char data, unsigned freq) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->left = temp->right = NULL;
    temp->data = data;
    temp->freq = freq;
    return temp;
}

void swap(struct Node** a, struct Node** b) {
    struct Node* t = *a;
    *a = *b;
    *b = t;
}

void heapify(struct MinHeap* heap, int i) {
    int smallest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if (left < heap->size && heap->array[left]->freq < heap->array[smallest]-
>freq)
        smallest = left;

    if (right < heap->size && heap->array[right]->freq < heap->array[smallest]-
>freq)
        smallest = right;

    if (smallest != i) {
        swap(&heap->array[i], &heap->array[smallest]);
        heapify(heap, smallest);
    }
}

struct Node* extractMin(struct MinHeap* heap) {
```

```
    struct Node* temp = heap->array[0];
    heap->array[0] = heap->array[heap->size - 1];
    heap->size--;
    heapify(heap, 0);
    return temp;
}

void insertHeap(struct MinHeap* heap, struct Node* node) {
    int i = heap->size;
    heap->size++;

    while (i && node->freq < heap->array[(i - 1)/2]->freq) {
        heap->array[i] = heap->array[(i - 1)/2];
        i = (i - 1)/2;
    }
    heap->array[i] = node;
}

struct Node* buildHuffman(char data[], int freq[], int n) {
    struct MinHeap heap;
    heap.size = n;

    for (int i = 0; i < n; i++) {
        heap.array[i] = newNode(data[i], freq[i]);
    }

    for (int i = (n-2)/2; i >= 0; i--) {
        heapify(&heap, i);
    }

    while (heap.size > 1) {
        struct Node* left = extractMin(&heap);
        struct Node* right = extractMin(&heap);

        struct Node* top = newNode('$', left->freq + right->freq);
        top->left = left;
        top->right = right;

        insertHeap(&heap, top);
    }

    return extractMin(&heap);
}

void printCodes(struct Node* root, int arr[], int top) {
    if (root->left) {
        arr[top] = 0;
        printCodes(root->left, arr, top + 1);
    }

    if (root->right) {
        arr[top] = 1;
        printCodes(root->right, arr, top + 1);
    }

    if (!root->left && !root->right) {
        printf("%c: ", root->data);
        for (int i = 0; i < top; i++)
            printf("%d", arr[i]);
        printf("\n");
    }
}

int main() {
```

```
    int n;

    printf("Enter number of characters: ");
    scanf("%d", &n);

    char data[n];
    int freq[n];

    printf("Enter characters:\n");
    for (int i = 0; i < n; i++)
        scanf(" %c", &data[i]);

    printf("Enter frequencies:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &freq[i]);

    struct Node* root = buildHuffman(data, freq, n);

    int arr[MAX], top = 0;

    printf("\nHuffman Codes:\n");
    printCodes(root, arr, top);

    return 0;
}
```

SLIP 6

Q-1) Write a program for the Implementation of Primâ ™s algorithm to find minimum cost spanning tree.

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

int minKey(int key[], int mstSet[], int V) {
    int min = INT_MAX, min_index = -1;

    for (int v = 0; v < V; v++) {
        if (mstSet[v] == 0 && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;
}

void printMST(int parent[], int graph[MAX][MAX], int V) {
    int total = 0;
    printf("\nEdge \tWeight\n");

    for (int i = 1; i < V; i++) {
        printf("%d - %d \t%d\n", parent[i], i, graph[i][parent[i]]);
        total += graph[i][parent[i]];
    }
```

```
    printf("Total Cost = %d\n", total);
}

void primMST(int graph[MAX][MAX], int V) {
    int parent[MAX];
    int key[MAX];
    int mstSet[MAX];

    for (int i = 0; i < V; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }

    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < V - 1; count++) {
        int u = minKey(key, mstSet, V);
        mstSet[u] = 1;

        for (int v = 0; v < V; v++) {
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    printMST(parent, graph, V);
}

int main() {
    int V;

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    int graph[MAX][MAX];

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    primMST(graph, V);

    return 0;
}
```

Q.2) Write a Program to find only length of Longest Common Subsequence.

```
#include <stdio.h>
#include <string.h>
#define MAX 100

int max(int a, int b) {
    return (a > b) ? a : b;
}
```

```
int main() {
    char X[MAX], Y[MAX];
    int m, n;

    printf("Enter first string: ");
    scanf("%s", X);

    printf("Enter second string: ");
    scanf("%s", Y);

    m = strlen(X);
    n = strlen(Y);

    int L[m + 1][n + 1];

    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0)
                L[i][j] = 0;
            else if (X[i - 1] == Y[j - 1])
                L[i][j] = L[i - 1][j - 1] + 1;
            else
                L[i][j] = max(L[i - 1][j], L[i][j - 1]);
        }
    }

    printf("Length of LCS = %d\n", L[m][n]);

    return 0;
}
```

SLIP 7

Q-1) Write a program for the Implementation of Dijkstraâ ™s algorithm to find shortest path to
other vertices

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

int minDistance(int dist[], int visited[], int V) {
    int min = INT_MAX, min_index = -1;

    for (int i = 0; i < V; i++) {
        if (!visited[i] && dist[i] <= min) {
            min = dist[i];
            min_index = i;
        }
    }
    return min_index;
}

void dijkstra(int graph[MAX][MAX], int V, int src) {
    int dist[MAX];
    int visited[MAX];
```

```
    for (int i = 0; i < V; i++) {
        dist[i] = INT_MAX;
        visited[i] = 0;
    }

    dist[src] = 0;

    for (int count = 0; count < V - 1; count++) {
        int u = minDistance(dist, visited, V);
        visited[u] = 1;

        for (int v = 0; v < V; v++) {
            if (!visited[v] && graph[u][v] &&
                dist[u] != INT_MAX &&
                dist[u] + graph[u][v] < dist[v]) {

                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printf("\nVertex \t Distance from Source\n");
    for (int i = 0; i < V; i++) {
        printf("%d \t %d\n", i, dist[i]);
    }
}

int main() {
    int V, src;

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    int graph[MAX][MAX];

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    printf("Enter source vertex: ");
    scanf("%d", &src);

    dijkstra(graph, V, src);

    return 0;
}
```

Q.2) Write a program for finding Topological sorting for Directed Acyclic Graph (DAG)

```
#include <stdio.h>

#define MAX 100

int main() {
    int V, E;
```

```
    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter number of edges: ");
    scanf("%d", &E);

    int adj[MAX][MAX] = {0};
    int indegree[MAX] = {0};

    printf("Enter edges (u v):\n");
    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
        indegree[v]++;
    }

    int queue[MAX], front = 0, rear = -1;

    for (int i = 0; i < V; i++) {
        if (indegree[i] == 0) {
            queue[++rear] = i;
        }
    }

    int count = 0;
    int topo[MAX];

    while (front <= rear) {
        int u = queue[front++];
        topo[count++] = u;

        for (int v = 0; v < V; v++) {
            if (adj[u][v]) {
                indegree[v]--;
                if (indegree[v] == 0) {
                    queue[++rear] = v;
                }
            }
        }
    }

    if (count != V) {
        printf("Graph is not a DAG (cycle exists)\n");
    } else {
        printf("Topological Order:\n");
        for (int i = 0; i < V; i++) {
            printf("%d ", topo[i]);
        }
        printf("\n");
    }

    return 0;
}
```

SLIP 8

Q.1) Write a program to implement Fractional Knapsack problems using Greedy

Method

```
#include <stdio.h>

struct Item {
    int value, weight;
    float ratio;
};

void sort(struct Item arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (arr[i].ratio < arr[j].ratio) {
                struct Item temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
}

int main() {
    int n, capacity;

    printf("Enter number of items: ");
    scanf("%d", &n);

    struct Item arr[n];

    printf("Enter values:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i].value);
    }

    printf("Enter weights:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i].weight);
        arr[i].ratio = (float)arr[i].value / arr[i].weight;
    }

    printf("Enter knapsack capacity: ");
    scanf("%d", &capacity);

    sort(arr, n);

    float totalValue = 0.0;

    for (int i = 0; i < n; i++) {
        if (capacity >= arr[i].weight) {
            capacity -= arr[i].weight;
            totalValue += arr[i].value;
        } else {
            totalValue += arr[i].ratio * capacity;
            break;
        }
    }

    printf("Maximum value in knapsack = %.2f\n", totalValue);

    return 0;
}
```

Q.2) Write Program to implement Traveling Salesman Problem using nearest neighbor
algorithm

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

int main() {
    int n;

    printf("Enter number of cities: ");
    scanf("%d", &n);

    int cost[MAX][MAX];

    printf("Enter cost matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }

    int visited[MAX] = {0};
    int path[MAX];
    int totalCost = 0;

    int current = 0;
    visited[current] = 1;
    path[0] = current;

    printf("\nPath: %d", current);

    for (int count = 1; count < n; count++) {
        int next = -1;
        int min = INT_MAX;

        for (int i = 0; i < n; i++) {
            if (!visited[i] && cost[current][i] != 0 && cost[current][i] < min)
{
                min = cost[current][i];
                next = i;
            }
        }

        if (next != -1) {
            visited[next] = 1;
            path[count] = next;
            totalCost += min;
            current = next;

            printf(" -> %d", next);
        }
    }

    totalCost += cost[current][0];
    printf(" -> %d", 0);

    printf("\nTotal Cost = %d\n", totalCost);

    return 0;
```

```
}
```

SLIP 9

Q.1) Write a program to implement optimal binary search tree and also calculate the best-casecomplexity.

```
#include <stdio.h>
#include <limits.h>

#define MAX 50

int min(int a, int b) {
    return (a < b) ? a : b;
}

int sum(int freq[], int i, int j) {
    int s = 0;
    for (int k = i; k <= j; k++)
        s += freq[k];
    return s;
}

int optimalBST(int keys[], int freq[], int n) {
    int cost[MAX][MAX];

    for (int i = 0; i < n; i++)
        cost[i][i] = freq[i];

    for (int L = 2; L <= n; L++) {
        for (int i = 0; i <= n - L; i++) {
            int j = i + L - 1;
            cost[i][j] = INT_MAX;

            for (int r = i; r <= j; r++) {
                int c = ((r > i) ? cost[i][r - 1] : 0) +
                        ((r < j) ? cost[r + 1][j] : 0) +
                        sum(freq, i, j);

                if (c < cost[i][j])
                    cost[i][j] = c;
            }
        }
    }

    return cost[0][n - 1];
}

int main() {
    int n;

    printf("Enter number of keys: ");
    scanf("%d", &n);

    int keys[n], freq[n];
```

```
    printf("Enter keys:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &keys[i]);

    printf("Enter frequencies:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &freq[i]);

    int cost = optimalBST(keys, freq, n);

    printf("Cost of Optimal BST = %d\n", cost);

    return 0;
}
```

Q.2) Write a program to implement Sum of Subset by Backtracking

```
#include <stdio.h>

#define MAX 50

int w[MAX], x[MAX];
int n, target;

void printSubset() {
    printf("{ ");
    for (int i = 0; i < n; i++) {
        if (x[i] == 1)
            printf("%d ", w[i]);
    }
    printf("}\n");
}

void sumOfSubsets(int k, int currentSum, int remainingSum) {
    x[k] = 1;

    if (currentSum + w[k] == target) {
        printSubset();
    }
    else if (currentSum + w[k] + remainingSum >= target && k + 1 < n) {
        sumOfSubsets(k + 1, currentSum + w[k], remainingSum - w[k]);
    }

    if (currentSum + remainingSum - w[k] >= target && k + 1 < n) {
        x[k] = 0;
        sumOfSubsets(k + 1, currentSum, remainingSum - w[k]);
    }
}

int main() {
    int total = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &w[i]);
        total += w[i];
    }
```

```
    printf("Enter target sum: ");
    scanf("%d", &target);

    printf("\nSubsets with sum = %d:\n", target);

    sumOfSubsets(0, 0, total);

    return 0;
}
```

SLIP 10

Q.1) Write a program to implement Huffman Code using greedy methods

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

struct Node {
    char data;
    int freq;
    struct Node *left, *right;
};

struct Node* newNode(char data, int freq) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->data = data;
    temp->freq = freq;
    temp->left = temp->right = NULL;
    return temp;
}

void swap(struct Node** a, struct Node** b) {
    struct Node* t = *a;
    *a = *b;
    *b = t;
}

void heapify(struct Node* heap[], int size, int i) {
    int smallest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if (left < size && heap[left]->freq < heap[smallest]->freq)
        smallest = left;

    if (right < size && heap[right]->freq < heap[smallest]->freq)
        smallest = right;

    if (smallest != i) {
        swap(&heap[i], &heap[smallest]);
        heapify(heap, size, smallest);
    }
}

struct Node* extractMin(struct Node* heap[], int *size) {
```

```
    struct Node* temp = heap[0];
    heap[0] = heap[*size - 1];
    (*size)--;
    heapify(heap, *size, 0);
    return temp;
}

void insertHeap(struct Node* heap[], int *size, struct Node* node) {
    int i = (*size)++;
    while (i && node->freq < heap[(i - 1) / 2]->freq) {
        heap[i] = heap[(i - 1) / 2];
        i = (i - 1) / 2;
    }
    heap[i] = node;
}

struct Node* buildHuffman(char data[], int freq[], int n) {
    struct Node* heap[MAX];
    int size = n;

    for (int i = 0; i < n; i++) {
        heap[i] = newNode(data[i], freq[i]);
    }

    for (int i = (size - 2) / 2; i >= 0; i--) {
        heapify(heap, size, i);
    }

    while (size > 1) {
        struct Node* left = extractMin(heap, &size);
        struct Node* right = extractMin(heap, &size);

        struct Node* top = newNode('$', left->freq + right->freq);
        top->left = left;
        top->right = right;

        insertHeap(heap, &size, top);
    }

    return heap[0];
}

void printCodes(struct Node* root, int arr[], int top) {
    if (root->left) {
        arr[top] = 0;
        printCodes(root->left, arr, top + 1);
    }

    if (root->right) {
        arr[top] = 1;
        printCodes(root->right, arr, top + 1);
    }

    if (!root->left && !root->right) {
        printf("%c: ", root->data);
        for (int i = 0; i < top; i++)
            printf("%d", arr[i]);
        printf("\n");
    }
}

int main() {
    int n;
```

```
    printf("Enter number of characters: ");
    scanf("%d", &n);

    char data[n];
    int freq[n];

    printf("Enter characters:\n");
    for (int i = 0; i < n; i++)
        scanf(" %c", &data[i]);

    printf("Enter frequencies:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &freq[i]);

    struct Node* root = buildHuffman(data, freq, n);

    int arr[MAX], top = 0;

    printf("\nHuffman Codes:\n");
    printCodes(root, arr, top);

    return 0;
}
```

Q-2) Write a program to solve 4 Queens Problem using Backtracking

```
#include <stdio.h>

#define N 4

int board[N][N];

int isSafe(int row, int col) {
    int i, j;

    for (i = 0; i < col; i++)
        if (board[row][i])
            return 0;

    for (i = row, j = col; i >= 0 && j >= 0; i--, j--)
        if (board[i][j])
            return 0;

    for (i = row, j = col; j >= 0 && i < N; i++, j--)
        if (board[i][j])
            return 0;

    return 1;
}

int solve(int col) {
    if (col >= N)
        return 1;

    for (int i = 0; i < N; i++) {
        if (isSafe(i, col)) {
            board[i][col] = 1;

            if (solve(col + 1))
                return 1;
```

```
            board[i][col] = 0;
        }
    }

    return 0;
}

void printSolution() {
    printf("Solution:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d ", board[i][j]);
        }
        printf("\n");
    }
}

int main() {

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            board[i][j] = 0;

    if (solve(0)) {
        printSolution();
    } else {
        printf("No solution exists\n");
    }

    return 0;
}
```

SLIP 11

Q.1) Write a programs to implement DFS (Depth First Search) and determine the time
complexity for the same.

```
#include <stdio.h>

#define MAX 100

int visited[MAX];

void DFS(int graph[MAX][MAX], int V, int v) {
    printf("%d ", v);
    visited[v] = 1;

    for (int i = 0; i < V; i++) {
        if (graph[v][i] == 1 && !visited[i]) {
            DFS(graph, V, i);
        }
    }
}

int main() {
```

```
    int V;

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    int graph[MAX][MAX];

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    int start;
    printf("Enter starting vertex: ");
    scanf("%d", &start);

    for (int i = 0; i < V; i++)
        visited[i] = 0;

    printf("DFS Traversal: ");
    DFS(graph, V, start);

    return 0;
}
```

Q.2) Write a program to find shortest paths from a given vertex in a weighted connected
graph, to other vertices using Dijkstraâ ™s algorithm.

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

int minDistance(int dist[], int visited[], int V) {
    int min = INT_MAX, min_index = -1;

    for (int i = 0; i < V; i++) {
        if (!visited[i] && dist[i] < min) {
            min = dist[i];
            min_index = i;
        }
    }
    return min_index;
}

void dijkstra(int graph[MAX][MAX], int V, int src) {
    int dist[MAX], visited[MAX];

    for (int i = 0; i < V; i++) {
        dist[i] = INT_MAX;
        visited[i] = 0;
    }

    dist[src] = 0;

    for (int count = 0; count < V - 1; count++) {
        int u = minDistance(dist, visited, V);
        visited[u] = 1;
```

```
        for (int v = 0; v < V; v++) {
            if (!visited[v] && graph[u][v] &&
                dist[u] != INT_MAX &&
                dist[u] + graph[u][v] < dist[v]) {

                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printf("\nVertex\tDistance from Source\n");
    for (int i = 0; i < V; i++) {
        printf("%d\t%d\n", i, dist[i]);
    }
}

int main() {
    int V, src;

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    int graph[MAX][MAX];

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    printf("Enter source vertex: ");
    scanf("%d", &src);

    dijkstra(graph, V, src);

    return 0;
}
```

SLIP 12

Q.1) Write a program to implement BFS (Breadth First Search) and determine the time
complexity for the same.

```
#include <stdio.h>

#define MAX 100

int queue[MAX], front = 0, rear = -1;
int visited[MAX];

void enqueue(int v) {
    queue[++rear] = v;
}
```

```
int dequeue() {
    return queue[front++];
}

int isEmpty() {
    return front > rear;
}

void BFS(int graph[MAX][MAX], int V, int start) {
    enqueue(start);
    visited[start] = 1;

    while (!isEmpty()) {
        int v = dequeue();
        printf("%d ", v);

        for (int i = 0; i < V; i++) {
            if (graph[v][i] == 1 && !visited[i]) {
                enqueue(i);
                visited[i] = 1;
            }
        }
    }
}

int main() {
    int V, start;

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    int graph[MAX][MAX];

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    printf("Enter starting vertex: ");
    scanf("%d", &start);

    for (int i = 0; i < V; i++)
        visited[i] = 0;

    printf("BFS Traversal: ");
    BFS(graph, V, start);

    return 0;
}
```

Q.2) Write a program to sort a given set of elements using the Selection sort method and
determine the time required to sort the elements.

```
#include <stdio.h>
#include <time.h>

// Selection Sort function
void selectionSort(int arr[], int n) {
```

```
    int i, j, min_idx, temp;

    for (i = 0; i < n - 1; i++) {
        min_idx = i;

        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }

        temp = arr[i];
        arr[i] = arr[min_idx];
        arr[min_idx] = temp;
    }
}

int main() {
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter elements:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    clock_t start, end;

    start = clock();
    selectionSort(arr, n);
    end = clock();

    printf("\nSorted array:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;

    printf("\nTime taken: %f seconds\n", time_taken);

    return 0;
}
```

SLIP 13

Q.1) Write a program to find minimum number of multiplications in Matrix Chain Multiplication.

```
#include <stdio.h>
#include <limits.h>

#define MAX 50
```

```
int matrixChainOrder(int p[], int n) {
    int m[MAX][MAX];

    for (int i = 1; i < n; i++)
        m[i][i] = 0;

    for (int L = 2; L < n; L++) {
        for (int i = 1; i < n - L + 1; i++) {
            int j = i + L - 1;
            m[i][j] = INT_MAX;

            for (int k = i; k < j; k++) {
                int q = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j];

                if (q < m[i][j])
                    m[i][j] = q;
            }
        }
    }

    return m[1][n - 1];
}

int main() {
    int n;

    printf("Enter number of matrices: ");
    scanf("%d", &n);

    int p[n + 1];

    printf("Enter dimensions array (size %d):\n", n + 1);
    for (int i = 0; i <= n; i++) {
        scanf("%d", &p[i]);
    }

    int result = matrixChainOrder(p, n + 1);

    printf("Minimum number of multiplications = %d\n", result);

    return 0;
}
```

Q.2) Write a program to implement an optimal binary search tree and also calculate the
best case and worst-case complexity.

```
#include <stdio.h>
#include <limits.h>

#define MAX 50

int sum(int freq[], int i, int j) {
    int s = 0;
    for (int k = i; k <= j; k++)
        s += freq[k];
    return s;
}

int optimalBST(int keys[], int freq[], int n) {
```

```
    int cost[MAX][MAX];

    for (int i = 0; i < n; i++)
        cost[i][i] = freq[i];

    for (int L = 2; L <= n; L++) {
        for (int i = 0; i <= n - L; i++) {
            int j = i + L - 1;
            cost[i][j] = INT_MAX;

            for (int r = i; r <= j; r++) {
                int c = ((r > i) ? cost[i][r - 1] : 0) +
                        ((r < j) ? cost[r + 1][j] : 0) +
                        sum(freq, i, j);

                if (c < cost[i][j])
                    cost[i][j] = c;
            }
        }
    }

    return cost[0][n - 1];
}

int main() {
    int n;

    printf("Enter number of keys: ");
    scanf("%d", &n);

    int keys[n], freq[n];

    printf("Enter keys:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &keys[i]);

    printf("Enter frequencies:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &freq[i]);

    int result = optimalBST(keys, freq, n);

    printf("Cost of Optimal BST = %d\n", result);

    return 0;
}
```

SLIP 14

Slip 14
Q.1) Write a program to sort a list of n numbers in ascending order using Insertion sort and
determine the time required to sort the elements.

```
#include <stdio.h>
#include <time.h>

void insertionSort(int arr[], int n) {
```

```
    int i, key, j;

    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }
}

int main() {
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter elements:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    clock_t start, end;

    start = clock();
    insertionSort(arr, n);
    end = clock();

    printf("\nSorted array:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;

    printf("\nTime taken: %f seconds\n", time_taken);

    return 0;
}
```

Q.2) Write a program to implement DFS and BFS. Compare the time complexity

```
#include <stdio.h>

#define MAX 100

int graph[MAX][MAX];
int visited[MAX];
int queue[MAX], front = 0, rear = -1;

void enqueue(int v) {
    queue[++rear] = v;
}

int dequeue() {
```

```
    return queue[front++];
}

int isEmpty() {
    return front > rear;
}

void BFS(int V, int start) {
    for (int i = 0; i < V; i++)
        visited[i] = 0;

    front = 0; rear = -1;

    enqueue(start);
    visited[start] = 1;

    printf("BFS Traversal: ");

    while (!isEmpty()) {
        int v = dequeue();
        printf("%d ", v);

        for (int i = 0; i < V; i++) {
            if (graph[v][i] == 1 && !visited[i]) {
                enqueue(i);
                visited[i] = 1;
            }
        }
    }
    printf("\n");
}

void DFS(int V, int v) {
    printf("%d ", v);
    visited[v] = 1;

    for (int i = 0; i < V; i++) {
        if (graph[v][i] == 1 && !visited[i]) {
            DFS(V, i);
        }
    }
}

int main() {
    int V, start;

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    printf("Enter starting vertex: ");
    scanf("%d", &start);

    BFS(V, start);

    for (int i = 0; i < V; i++)
        visited[i] = 0;
```

```
    printf("DFS Traversal: ");
    DFS(V, start);
    printf("\n");

    return 0;
}
```

SLIP 15

Q.1) Write a program to implement to find out solution for 0/1 knapsack problem using
LCBB (Least Cost Branch and Bound).

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

struct Node {
    int level, profit, weight;
    float bound;
};

struct Item {
    int value, weight;
    float ratio;
};

void sort(struct Item arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (arr[i].ratio < arr[j].ratio) {
                struct Item temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
}

float bound(struct Node u, int n, int W, struct Item arr[]) {
    if (u.weight >= W)
        return 0;

    float profit_bound = u.profit;
    int j = u.level + 1;
    int totweight = u.weight;

    while (j < n && totweight + arr[j].weight <= W) {
        totweight += arr[j].weight;
        profit_bound += arr[j].value;
        j++;
    }

    if (j < n) {
        profit_bound += (W - totweight) * arr[j].ratio;
    }
```

```
    return profit_bound;
}

int knapsack(int W, struct Item arr[], int n) {
    struct Node u, v;
    struct Node queue[MAX];
    int front = 0, rear = 0;

    int maxProfit = 0;

    v.level = -1;
    v.profit = 0;
    v.weight = 0;
    v.bound = bound(v, n, W, arr);

    queue[rear++] = v;

    while (front < rear) {
        v = queue[front++];

        if (v.level == n - 1)
            continue;

        u.level = v.level + 1;
        u.weight = v.weight + arr[u.level].weight;
        u.profit = v.profit + arr[u.level].value;

        if (u.weight <= W && u.profit > maxProfit)
            maxProfit = u.profit;

        u.bound = bound(u, n, W, arr);

        if (u.bound > maxProfit)
            queue[rear++] = u;

        u.weight = v.weight;
        u.profit = v.profit;
        u.bound = bound(u, n, W, arr);

        if (u.bound > maxProfit)
            queue[rear++] = u;
    }

    return maxProfit;
}

int main() {
    int n, W;

    printf("Enter number of items: ");
    scanf("%d", &n);

    struct Item arr[n];

    printf("Enter values:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i].value);

    printf("Enter weights:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i].weight);
        arr[i].ratio = (float)arr[i].value / arr[i].weight;
    }
```

```
    printf("Enter knapsack capacity: ");
    scanf("%d", &W);

    sort(arr, n);

    int result = knapsack(W, arr, n);

    printf("Maximum profit = %d\n", result);

    return 0;
}
```

Q.2) Write a program to implement Graph Coloring Algorithm

```
#include <stdio.h>

#define MAX 100

int graph[MAX][MAX];
int color[MAX];
int V, m;

int isSafe(int v, int c) {
    for (int i = 0; i < V; i++) {
        if (graph[v][i] == 1 && color[i] == c)
            return 0;
    }
    return 1;
}

int graphColoring(int v) {
    if (v == V)
        return 1;

    for (int c = 1; c <= m; c++) {
        if (isSafe(v, c)) {
            color[v] = c;

            if (graphColoring(v + 1))
                return 1;

            color[v] = 0;
        }
    }
    return 0;
}

int main() {
    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    printf("Enter number of colors: ");
    scanf("%d", &m);

    for (int i = 0; i < V; i++)
```

```
        color[i] = 0;

    if (graphColoring(0)) {
        printf("Solution exists:\n");
        for (int i = 0; i < V; i++) {
            printf("Vertex %d -> Color %d\n", i, color[i]);
        }
    } else {
        printf("No solution exists\n");
    }

    return 0;
}
```

SLIP 16

Q.1) Write a program to implement to find out solution for 0/1 knapsack problem using
dynamic programming.

```
#include <stdio.h>
#define MAX 100

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n, W;

    printf("Enter number of items: ");
    scanf("%d", &n);

    int wt[n], val[n];

    printf("Enter weights:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &wt[i]);

    printf("Enter values:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &val[i]);

    printf("Enter knapsack capacity: ");
    scanf("%d", &W);

    int dp[n + 1][W + 1];

    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (wt[i - 1] <= w)
                dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]],
                               dp[i - 1][w]);
            else
                dp[i][w] = dp[i - 1][w];
```

```
        }
    }

    printf("Maximum value = %d\n", dp[n][W]);

    return 0;
}
```

Q.2) Write a program to determine if a given graph is a Hamiltonian cycle or not.

```
#include <stdio.h>
#define MAX 100

int graph[MAX][MAX];
int path[MAX];
int V;

int isSafe(int v, int pos) {

    if (graph[path[pos - 1]][v] == 0)
        return 0;

    for (int i = 0; i < pos; i++) {
        if (path[i] == v)
            return 0;
    }

    return 1;
}

int hamCycleUtil(int pos) {

    if (pos == V) {

        return graph[path[pos - 1]][path[0]] == 1;
    }

    for (int v = 1; v < V; v++) {
        if (isSafe(v, pos)) {
            path[pos] = v;

            if (hamCycleUtil(pos + 1))
                return 1;

            path[pos] = -1;
        }
    }

    return 0;
}

void hamCycle() {
    for (int i = 0; i < V; i++)
        path[i] = -1;

    path[0] = 0;

    if (!hamCycleUtil(1)) {
        printf("No Hamiltonian Cycle exists\n");
        return;
```

```
    }

    printf("Hamiltonian Cycle:\n");
    for (int i = 0; i < V; i++)
        printf("%d ", path[i]);

    printf("%d\n", path[0]);
}

int main() {
    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    hamCycle();

    return 0;
}
```

SLIP 17

Q.1) Write a program to implement solve â ˜Nâ ™ Queens Problem using Backtracking.

```
#include <stdio.h>
#define MAX 20

int board[MAX][MAX];
int N;

int isSafe(int row, int col) {
    int i, j;

    for (i = 0; i < row; i++)
        if (board[i][col])
            return 0;

    for (i = row, j = col; i >= 0 && j >= 0; i--, j--)
        if (board[i][j])
            return 0;

    for (i = row, j = col; i >= 0 && j < N; i--, j++)
        if (board[i][j])
            return 0;

    return 1;
}

int solve(int row) {
    if (row == N)
        return 1;
```

```
    for (int col = 0; col < N; col++) {
        if (isSafe(row, col)) {
            board[row][col] = 1;

            if (solve(row + 1))
                return 1;

            board[row][col] = 0;
        }
    }
    return 0;
}

void printSolution() {
    printf("Solution:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d ", board[i][j]);
        }
        printf("\n");
    }
}

int main() {
    printf("Enter value of N: ");
    scanf("%d", &N);

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            board[i][j] = 0;

    if (solve(0))
        printSolution();
    else
        printf("No solution exists\n");

    return 0;
}
```

Q.2) Write a program to find out solution for 0/1 knapsack problem.

```
#include <stdio.h>
#define MAX 100

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n, W;

    printf("Enter number of items: ");
    scanf("%d", &n);

    int wt[n], val[n];

    printf("Enter weights:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &wt[i]);

    printf("Enter values:\n");
```

```
    for (int i = 0; i < n; i++)
        scanf("%d", &val[i]);

    printf("Enter knapsack capacity: ");
    scanf("%d", &W);

    int dp[n + 1][W + 1];

    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (wt[i - 1] <= w)
                dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]],
                               dp[i - 1][w]);
            else
                dp[i][w] = dp[i - 1][w];
        }
    }

    printf("Maximum value = %d\n", dp[n][W]);

    return 0;
}
```

SLIP 18

Q.1) Write a program to implement Graph Coloring Algorithm

```
#include <stdio.h>
#define MAX 100

int graph[MAX][MAX];
int color[MAX];
int V, m;

int isSafe(int v, int c) {
    for (int i = 0; i < V; i++) {
        if (graph[v][i] == 1 && color[i] == c)
            return 0;
    }
    return 1;
}

int solve(int v) {
    if (v == V)
        return 1;

    for (int c = 1; c <= m; c++) {
        if (isSafe(v, c)) {
            color[v] = c;

            if (solve(v + 1))
                return 1;

            color[v] = 0;
```

```
        }
    }
    return 0;
}

int main() {
    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    printf("Enter number of colors: ");
    scanf("%d", &m);

    for (int i = 0; i < V; i++)
        color[i] = 0;

    if (solve(0)) {
        printf("Solution exists:\n");
        for (int i = 0; i < V; i++) {
            printf("Vertex %d -> Color %d\n", i, color[i]);
        }
    } else {
        printf("No solution exists\n");
    }

    return 0;
}
```

Q.2) Write a program to find out live node, E node and dead node from a given graph.

```
#include <stdio.h>
#define MAX 10

int graph[MAX][MAX], visited[MAX];
int queue[MAX];
int front = 0, rear = -1;

void enqueue(int x) {
    queue[++rear] = x;
}

int dequeue() {
    return queue[front++];
}

int isEmpty() {
    return front > rear;
}

int main() {
    int V, start;

    printf("Enter number of vertices: ");
```

```
    scanf("%d", &V);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            scanf("%d", &graph[i][j]);

    printf("Enter starting node: ");
    scanf("%d", &start);

    // Initialize
    for (int i = 0; i < V; i++)
        visited[i] = 0;

    enqueue(start);
    visited[start] = 1;

    printf("\nTraversal with Node Types:\n");

    while (!isEmpty()) {
        int node = dequeue();

        printf("E-node (Expanding): %d\n", node);

        int isLeaf = 1;

        for (int i = 0; i < V; i++) {
            if (graph[node][i] == 1 && !visited[i]) {
                enqueue(i);
                visited[i] = 1;

                printf("Live node: %d\n", i);
                isLeaf = 0;
            }
        }

        if (isLeaf) {
            printf("Dead node: %d\n", node);
        }

        printf("\n");
    }

    return 0;
}
```

SLIP 19

Q.1) Write a program to determine if a given graph is a Hamiltonian cycle or not.

```
#include <stdio.h>
#define MAX 100

int graph[MAX][MAX];
int path[MAX];
```

```
int V;

int isSafe(int v, int pos) {

    if (graph[path[pos - 1]][v] == 0)
        return 0;

    for (int i = 0; i < pos; i++) {
        if (path[i] == v)
            return 0;
    }

    return 1;
}

int hamCycleUtil(int pos) {

    if (pos == V) {

        return graph[path[pos - 1]][path[0]] == 1;
    }

    for (int v = 1; v < V; v++) {
        if (isSafe(v, pos)) {
            path[pos] = v;

            if (hamCycleUtil(pos + 1))
                return 1;

            path[pos] = -1;
        }
    }

    return 0;
}

void hamCycle() {
    for (int i = 0; i < V; i++)
        path[i] = -1;

    path[0] = 0;

    if (!hamCycleUtil(1)) {
        printf("No Hamiltonian Cycle exists\n");
        return;
    }

    printf("Hamiltonian Cycle:\n");
    for (int i = 0; i < V; i++)
        printf("%d ", path[i]);

    printf("%d\n", path[0]);
}

int main() {
    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
```

```
    hamCycle();

    return 0;
}
```

Q.2) Write a program to show board configuration of 4 queensâ ™ problem.

```
#include <stdio.h>
#define N 4

int board[N][N];

int isSafe(int row, int col) {
    int i, j;

    for (i = 0; i < col; i++)
        if (board[row][i])
            return 0;

    for (i = row, j = col; i >= 0 && j >= 0; i--, j--)
        if (board[i][j])
            return 0;

    for (i = row, j = col; i < N && j >= 0; i++, j--)
        if (board[i][j])
            return 0;

    return 1;
}

int solve(int col) {
    if (col >= N)
        return 1;

    for (int i = 0; i < N; i++) {
        if (isSafe(i, col)) {
            board[i][col] = 1;

            if (solve(col + 1))
                return 1;

            board[i][col] = 0;
        }
    }
    return 0;
}

void printBoard() {
    printf("Board Configuration (1 = Queen, 0 = Empty):\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d ", board[i][j]);
        }
        printf("\n");
    }
}

int main() {
```

```
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            board[i][j] = 0;

    if (solve(0)) {
        printBoard();
    } else {
        printf("No solution exists\n");
    }

    return 0;
}
```

_____-

SLIP 20

Q.1) Write a program to implement for finding Topological sorting and determine the time
complexity for the same.

```
#include <stdio.h>
#define MAX 100

int graph[MAX][MAX];
int indegree[MAX];
int queue[MAX], front = 0, rear = -1;

void enqueue(int x) {
    queue[++rear] = x;
}

int dequeue() {
    return queue[front++];
}

int isEmpty() {
    return front > rear;
}

int main() {
    int V;

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    for (int i = 0; i < V; i++) {
        indegree[i] = 0;
        for (int j = 0; j < V; j++) {
            if (graph[j][i] == 1)
                indegree[i]++;
        }
    }
```

```
    for (int i = 0; i < V; i++) {
        if (indegree[i] == 0)
            enqueue(i);
    }

    printf("Topological Order: ");

    int count = 0;

    while (!isEmpty()) {
        int v = dequeue();
        printf("%d ", v);

        count++;

        for (int i = 0; i < V; i++) {
            if (graph[v][i] == 1) {
                indegree[i]--;
                if (indegree[i] == 0)
                    enqueue(i);
            }
        }
    }

    if (count != V) {
        printf("\nGraph is not a DAG (cycle exists)\n");
    }

    return 0;
}
```

Q.2) Write a program to solve N Queens Problem using Backtracking.

```
#include <stdio.h>
#define MAX 20

int board[MAX][MAX];
int N;

int isSafe(int row, int col) {
    int i, j;

    for (i = 0; i < row; i++)
        if (board[i][col])
            return 0;

    for (i = row, j = col; i >= 0 && j >= 0; i--, j--)
        if (board[i][j])
            return 0;

    for (i = row, j = col; i >= 0 && j < N; i--, j++)
        if (board[i][j])
            return 0;

    return 1;
}

int solve(int row) {
    if (row == N)
        return 1;
```

```
    for (int col = 0; col < N; col++) {
        if (isSafe(row, col)) {
            board[row][col] = 1;

            if (solve(row + 1))
                return 1;

            board[row][col] = 0;
        }
    }
    return 0;
}

void printSolution() {
    printf("Solution:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d ", board[i][j]);
        }
        printf("\n");
    }
}

int main() {
    printf("Enter value of N: ");
    scanf("%d", &N);

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            board[i][j] = 0;

    if (solve(0))
        printSolution();
    else
        printf("No solution exists\n");

    return 0;
}
```

_____
*****_____